ZAINA RAHEEN A P

241801328

BUILT-IN METHODS / FUNCTIONS USED AND THEIR PURPOSE

Function / Method	Purpose
`pd.read_csv()`	Reads the CSV dataset
`parse_dates`	Converts the Month column into dates
`index_col`	Makes Month the time-series index
`head()`	Displays first few records
`info()`	Displays dataset information
`describe()`	Gives statistical summary
`shape`	Gives number of rows and columns
`diff()`	Performs differencing
`dropna()`	Removes missing values
`adfuller()`	Performs the Augmented Dickey-Fuller stationarity test
`ARIMA()`	Creates the ARIMA forecasting model
`fit()`	Trains the ARIMA model
`forecast()`	Generates future predictions
`mean_squared_error()`	Calculates MSE
`mean_absolute_error()`	Calculates MAE
`np.sqrt()`	Calculates RMSE
`plt.plot()`	Plots the time series
`plt.legend()`	Displays plot labels

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

from sklearn.metrics import mean_squared_error, mean_absolute_error
```

```
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

df = pd.read_csv(
    url,
    parse_dates=["Month"],
    index_col="Month"
)

df.head()
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

```
print("Dataset Shape:", df.shape)

print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
print(df.describe())

print("\nFirst 5 Rows:")
print(df.head())
```

```
Dataset Shape: (144, 1)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB

Statistical Summary:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000

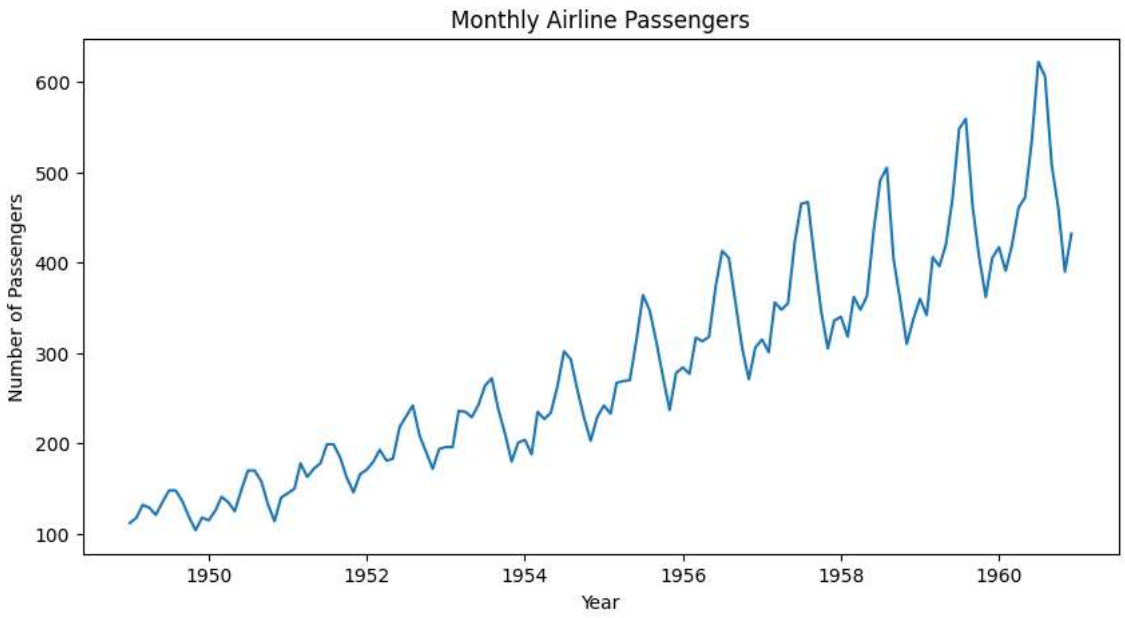
First 5 Rows:
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
```

```
plt.figure(figsize=(10, 5))

plt.plot(df["Passengers"])

plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")

plt.show()
```




```
result = adfuller(df["Passengers"])

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] <= 0.05:
    print("The series is stationary")
else:
    print("The series is non-stationary")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
The series is non-stationary
/tmp/ipykernel_1269/3587247533.py:1: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store a
  result = adfuller(df["Passengers"])
```

```
df["Differenced"] = df["Passengers"].diff()

print(df.head())
```

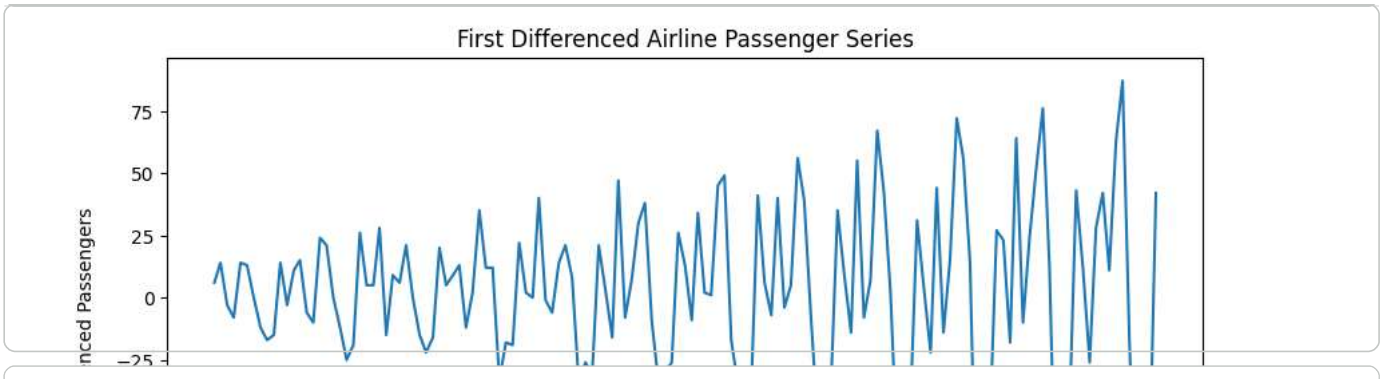
```
            Passengers  Differenced
Month
1949-01-01         112          NaN
1949-02-01         118          6.0
1949-03-01         132         14.0
1949-04-01         129         -3.0
1949-05-01         121         -8.0
```

```
plt.figure(figsize=(10, 5))

plt.plot(df["Differenced"])

plt.title("First Differenced Airline Passenger Series")
plt.xlabel("Year")
plt.ylabel("Differenced Passengers")

plt.show()
```




```
result = adfuller(df["Differenced"].dropna())

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] <= 0.05:
    print("The differenced series is stationary")
else:
    print("The differenced series is non-stationary")
```

```
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
The differenced series is non-stationary
/tmp/ipykernel_1269/3984137407.py:1: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store a
  result = adfuller(df["Differenced"].dropna())
```

```
series = df["Passengers"]

train_size = int(len(series) * 0.8)

train = series.iloc[:train_size]
test = series.iloc[train_size:]

print("Training Data:")
print(train.tail())

print("\nTesting Data:")
print(test.head())
```

```
Training Data:
Month
1958-03-01    362
1958-04-01    348
1958-05-01    363
1958-06-01    435
1958-07-01    491
Name: Passengers, dtype: int64

Testing Data:
Month
1958-08-01    505
1958-09-01    404
1958-10-01    359
1958-11-01    310
1958-12-01    337
Name: Passengers, dtype: int64
```

```
model = ARIMA(train, order=(1, 1, 1))

model_fit = model.fit()

print(model_fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provid
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provid
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provid
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(1, 1, 1)   Log Likelihood                -526.123
```

```
Date:                Wed, 23 Sep 2026   AIC                           1058.246
Time:                        06:34:52   BIC                           1066.454
Sample:                  01-01-1949   HQIC                          1061.577
                       - 07-01-1958
Covariance Type:                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5111      0.114     -4.488      0.000      -0.734      -0.288
ma.L1          0.9144      0.056     16.251      0.000       0.804       1.025
sigma2       592.7851    101.208      5.857      0.000     394.422     791.148
===================================================================================
Ljung-Box (L1) (Q):                   0.09   Jarque-Bera (JB):                 2.29
Prob(Q):                              0.76   Prob(JB):                         0.32
Heteroskedasticity (H):               5.22   Skew:                             0.03
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.31
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
forecast = model_fit.forecast(steps=len(test))

forecast.index = test.index

print("Predicted Passenger Numbers:")
print(forecast)
```

```
Predicted Passenger Numbers:
Month
1958-08-01    502.649601
1958-09-01    496.695602
1958-10-01    499.738634
1958-11-01    498.183370
1958-12-01    498.978250
1959-01-01    498.571994
1959-02-01    498.779628
1959-03-01    498.673508
1959-04-01    498.727745
1959-05-01    498.700025
1959-06-01    498.714192
1959-07-01    498.706952
1959-08-01    498.710652
1959-09-01    498.708761
1959-10-01    498.709728
1959-11-01    498.709234
1959-12-01    498.709486
1960-01-01    498.709357
1960-02-01    498.709423
1960-03-01    498.709389
1960-04-01    498.709407
1960-05-01    498.709398
1960-06-01    498.709402
1960-07-01    498.709400
1960-08-01    498.709401
1960-09-01    498.709400
1960-10-01    498.709401
1960-11-01    498.709401
1960-12-01    498.709401
Name: predicted_mean, dtype: float64
```

```
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)

print("MSE :", mse)
print("MAE :", mae)
print("RMSE:", rmse)
```

```
MSE : 9506.17584193304
MAE : 85.25415598223435
RMSE: 97.49961970147903
```

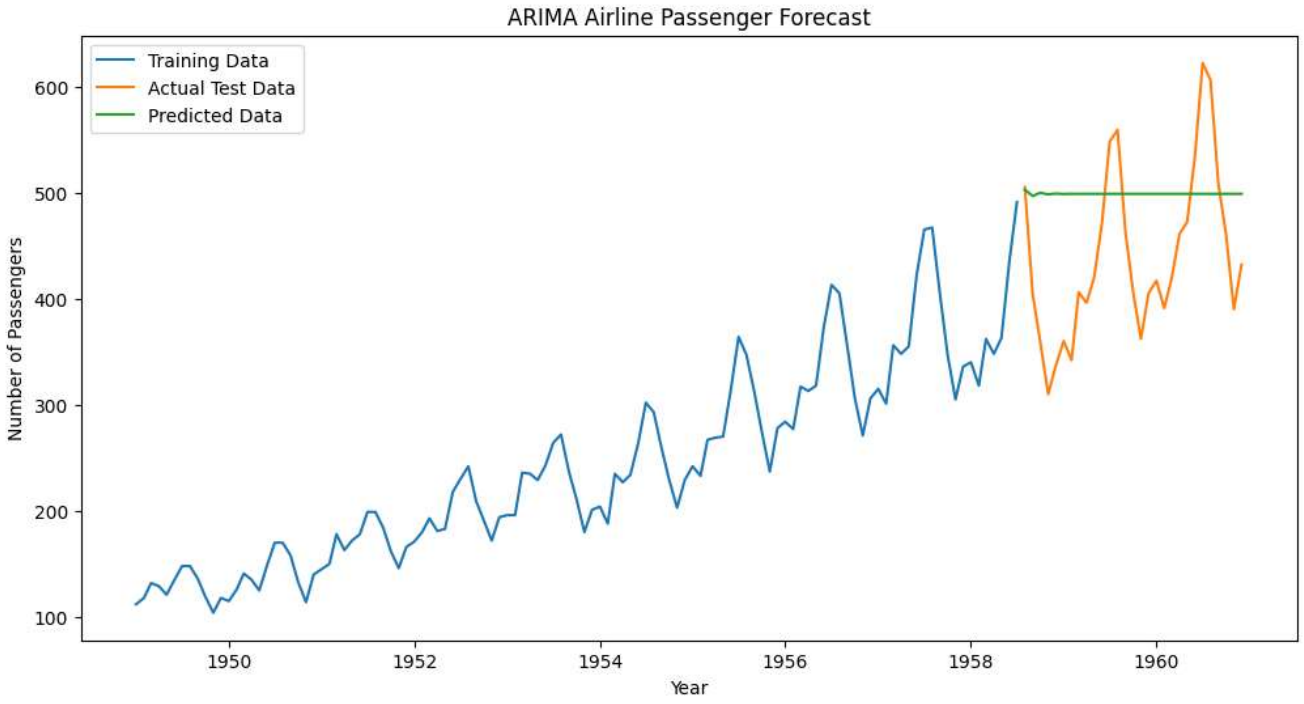
```
plt.figure(figsize=(12, 6))

plt.plot(train, label="Training Data")
plt.plot(test, label="Actual Test Data")
plt.plot(forecast, label="Predicted Data")

plt.title("ARIMA Airline Passenger Forecast")
```

```
plt.xlabel("Year")
plt.ylabel("Number of Passengers")

plt.legend()
plt.show()
```




```
comparison = pd.DataFrame({
    "Actual": test,
    "Predicted": forecast
})

print(comparison)
```

```
            Actual   Predicted
Month
1958-08-01     505  502.649601
1958-09-01     404  496.695602
1958-10-01     359  499.738634
1958-11-01     310  498.183370
1958-12-01     337  498.978250
1959-01-01     360  498.571994
1959-02-01     342  498.779628
1959-03-01     406  498.673508
1959-04-01     396  498.727745
1959-05-01     420  498.700025
1959-06-01     472  498.714192
1959-07-01     548  498.706952
1959-08-01     559  498.710652
1959-09-01     463  498.708761
1959-10-01     407  498.709728
1959-11-01     362  498.709234
1959-12-01     405  498.709486
1960-01-01     417  498.709357
1960-02-01     391  498.709423
1960-03-01     419  498.709389
1960-04-01     461  498.709407
1960-05-01     472  498.709398
1960-06-01     535  498.709402
1960-07-01     622  498.709400
1960-08-01     606  498.709401
1960-09-01     508  498.709400
1960-10-01     461  498.709401
1960-11-01     390  498.709401
1960-12-01     432  498.709401
```

Learning Outcome

Students will be able to analyze time series data, check stationarity, apply differencing, build an ARIMA model, forecast future values, and evaluate prediction accuracy using MSE, MAE, and RMSE.

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```